

NumPy Complete Notes (Beginner to Advanced)

With Student FAQs + Interview Qs

By: Rimsha Shahzadi



Table of Contents (Optional)

1. NumPy Introduction
2. Array Creation
3. Array Attributes
4. Indexing & Slicing
5. Array Operations
6. Array Reshaping
7. Boolean Indexing & Filtering
8. Stacking & Splitting
9. Advanced Topics
10. NaN & Inf Handling

✿ 1. NumPy ka Introduction

NumPy kya hai?

NumPy (Numerical Python) ek Python library hai jo fast mathematical operations ke liye use hoti hai. Ye specially arrays (ndarrays) ke sath kaam karti hai. Ye large data ko efficiently handle karti hai jo normal Python lists se bahut fast hoti hai.

Install & Import

```
pip install numpy
```

```
import numpy as np
```

Student FAQs

Q: NumPy list se fast kyun hoti hai?

A: Kyunki NumPy C language me likhi gayi hai aur contiguous memory allocation use karti hai, jabke Python lists scattered memory use karti hain.

Q: NumPy ka full form kya hai?

A: Numerical Python

Interview Qs + Answers

Q: NumPy kya hai aur Python list se fast kyun hai?

A: NumPy numerical computation ke liye optimized library hai. Ye fixed-type arrays use karti hai jo contiguous memory me store hotay hain, isliye Python lists se fast hoti hai.

Q: NumPy ka data structure kya hota hai?

A: NumPy ka main data structure ndarray (N-dimensional array) hota hai.

Q: NumPy ko import karte waqt as np kyun likhte hain?

A: Taake har jagah numpy likhne ke bajaye short form np likh kar use kar saken.

2. Array Creation (ndarray banana)

Common Functions

Function ka	Kaam
-------------	------

np.array():	Python list ko ndarray me convert karta hai
--------------------	---

np.zeros(shape):	0s se filled array
-------------------------	--------------------

np.ones(shape):	1s se filled array
------------------------	--------------------

np.full(shape,val):	Specific value se filled array
----------------------------	--------------------------------

np.eye(n): Identity matrix

np.arange(start,stop,step): Range ke numbers

np.linspace(start,stop,num): Equal spaced values

np.random.randint(low,high,size): Random integers

np.random.rand(shape): Random floats (0–1)

Examples

```
a = np.array([1,2,3])
```

```
b = np.zeros((2,2))
```

```
c = np.ones((3,3))
```

```
d = np.eye(3)
```

```
e = np.arange(1,10,2)
```

```
f = np.linspace(0,1,5)
```

Student FAQs

Q: arange aur linspace me kya farq hai?

A: arange step size se values deta hai, linspace equal number of points deta hai.

Q: random.rand aur random.randint me kya farq hai?

A: rand float values deta hai 0–1 ke beech, randint integer values deta hai.

Interview Qs + Answers

Q: NumPy array banane ke 5 tareeqe batao?

A: np.array(), np.zeros(), np.ones(), np.arange(), np.random.randint()

Q: zeros aur empty me kya farq hai?

A: zeros() array me sab elements 0 set karta hai. empty() sirf memory allocate karta hai (values random garbage ho sakti hain).

Q: Identity matrix kya hoti hai?

A: Square matrix jisme diagonal pe sab 1 hote hain aur baaki sab 0 — np.eye(n) se banti hai.

3. Array Attributes

Attribute ke Kaam

.ndim: Dimensions count

.shape: Rows × Columns

.size: Total elements

.dtype: Data type

.itemsize: Har element ka size (bytes)

.nbytes: Total memory (bytes)

Example

```
a = np.array([[1,2,3],[4,5,6]])
```

```
print(a.ndim)    # 2
```

```
print(a.shape)    # (2,3)
```

```
print(a.size)    # 6
```

```
print(a.dtype)    # int64
```

Student FAQs

Q: ndim aur shape me kya farq hai?

A: ndim sirf dimension count dikhata hai, shape har dimension me kitne elements hain dikhata hai.

Interview Qs + Answers

Q: NumPy array ka size kaise check karte hain?

A: array.size use karke.

Q: Data type kaise dekhte hain?

A: array.dtype se.

4. Indexing & Slicing

Indexing: Specific element ko access karna

Slicing: Range of elements ko access karna

Example

```
a = np.array([10,20,30,40,50])
```

```
print(a[0])    # 10
```

```
print(a[1:4])  # [20 30 40]
```

```
b = np.array([[1,2,3],[4,5,6]])
```

```
print(b[0,1]) # 2
```

```
print(b[:,1]) # 2nd column
```

Student FAQs

Q: Slicing original array ko affect karti hai?

A: Haan, slicing view deti hai jo original array ko affect kar sakti hai.

Interview Qs + Answers

Q: Slicing aur Indexing me farq?

A: Indexing ek element return karti hai, slicing multiple elements return karti hai.

Q: Negative indexing kya hoti hai?

A: Jab end se elements access karte hain (-1 last element hota hai).

5. Array Operations (Math)

Operation ke Kaam

+ - * / ** Element-wise operations

np.add(): Subtraction

np.multiply(): Multiplication

np.divide(): Division

np.sqrt(): Square root

np.sum(): Sum

np.mean(): Mean

np.max() / np.min(): Max/Min

np.argmax() / np.argmin(): Max/Min index

Example

```
a = np.array([1,2,3])
```

```
b = np.array([4,5,6])
```

```
print(a+b)
```

```
print(np.sqrt(a))
```

```
print(np.sum(b))
```

Student FAQs

Q: Broadcasting kya hota hai?

Definition: Jab 2 arrays ke shapes match nahi karte to NumPy chote array ko automatically expand karta hai taake dono same shape ke ban jayein.

Explanation :

Broadcasting NumPy ka ek powerful feature hai jo **different shapes ke arrays par operations karne** ki facility deta hai — bina unhe manually resize ya repeat kiye.

Normally, mathematical operations (jaise + - * /) tabhi kaam karte hain jab arrays ka **same shape** ho. Lekin broadcasting chhote array ko **automatically bada (expand)** karke compatible bana deta hai.

Broadcasting Rules

NumPy broadcasting tabhi possible hoti hai jab ye rules satisfy hon:

1. **Dimensions Aligning (Right se)**

Compare dimensions from **right side se left** ki taraf.

2. **Equal or 1 Size**

Har axis ke corresponding dimension **ya to same hon, ya kisi ek array me 1 ho**.

3. **Dimension Expand Hoti Hai**

Agar kisi axis ka size 1 hai, to NumPy us array ko **repeat** karke dusre array ke size jaisa treat karta hai.

4. Incompatible Shape → Error

Agar koi dimension na same hai na 1, to **ValueError** aata hai.

⚙️ Broadcasting Functions / Methods:

Broadcasting ke liye koi specific method nahi hoti — ye **automatically apply hoti hai** jab tum mathematical operators use karte ho. Lekin kuch helper functions hain jo broadcasting ke sath useful hote hain:

Function	Purpose
<code>np.add, np.subtract, np.multiply, np.divide</code>	Element-wise operations (broadcasting apply hoti hai)
<code>np.broadcast_to(arr, shape)</code>	Array ko explicitly broadcast shape me convert karta hai
<code>np.expand_dims(arr, axis)</code>	Extra dimension add karta hai broadcasting ke compatible banane ke liye
<code>np.newaxis</code>	Array me new axis add kar deta hai (reshape ke liye)

💻 Examples

Example 1: 1D + 1D (same shape)

```
a = np.array([1,2,3])
b = np.array([10,20,30])
print(a + b)
```

Output: [11 22 33]

Example 2: 1D + 2D (broadcasting)

```
x = np.array([[1],[2],[3]])
y = np.array([10,20,30])
print(x + y)
```


Output:

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

Example 3: Using broadcast_to:

```
arr = np.array([1,2,3])
print(np.broadcast_to(arr, (3,3)))
```

Output:

```
[[1 2 3]
 [1 2 3]
 [1 2 3]]
```

Example 4: Using np.newaxis to enable broadcasting:

```
a = np.array([1,2,3])
b = np.array([10,20,30])
print(a[:, np.newaxis] + b)
```

Output:

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

? Possible Questions (FAQs):**Q: reshape aur resize me farq?**

A: reshape view deta hai (original nahi badalti), resize original array ko badal deta hai.

Interview Qs + Answers

Q: Transpose kya hota hai?

A: Jab rows aur columns ko swap kar diya jaye (.T).

Q: Flatten aur ravel me farq?

A: ravel view return karta hai (memory share hoti hai), flatten copy return karta hai (memory alag hoti hai).

 7. Boolean Indexing & Filtering:

 Explanation:

Boolean indexing ka matlab hai condition laga kar specific rows/columns ko filter karna. Condition True hoti hai to woh row/element select hota hai, warna ignore ho jata hai.

 Common Methods / Techniques

Method / Approach	Kaam (Purpose)
df[condition]	Condition ke sath rows filter karna
df[df['col'] > value]	Column par condition laga kar filter karna
df.loc[condition]	Label-based filtering
df.iloc[condition]	Position-based filtering
df.query("condition")	String query syntax se filtering
Multiple conditions (&, `~`)	
isin(), between(), str.contains()	Special conditions lagane ke liye

 Examples

Sample DataFrame

```
data = {  
    'Name': ['Ali','Sara','Ahmed','Hina','Zara'],  
    'Age': [25, 30, 22, 28, 24],  
    'City': ['Lahore','Karachi','Lahore','Islamabad','Karachi']  
}  
  
df = pd.DataFrame(data)
```

1. Basic boolean condition

```
print(df[df['Age'] > 25])
```

Rows jinki Age 25 se zyada hai

2. Multiple conditions (AND: &)

```
print(df[(df['Age'] > 23) & (df['City'] == 'Karachi')])
```

3. Multiple conditions (OR: |)

```
print(df[(df['City'] == 'Lahore') | (df['City'] == 'Islamabad')])
```

4. NOT condition (~)

```
print(df[~(df['City'] == 'Lahore')])
```

5. Using .query()

```
print(df.query("Age < 28 and City == 'Karachi'"))
```

6. Using .isin()

```
print(df[df['City'].isin(['Karachi','Islamabad'])])
```

7. Using .between()

```
print(df[df['Age'].between(23,28)])
```

Output:

	Name	Age	City
1	Sara	30	Karachi
3	Hina	28	Islamabad

	Name	Age	City
1	Sara	30	Karachi
4	Zara	24	Karachi

	Name	Age	City
0	Ali	25	Lahore
2	Ahmed	22	Lahore
3	Hina	28	Islamabad

	Name	Age	City
1	Sara	30	Karachi
3	Hina	28	Islamabad
4	Zara	24	Karachi

	Name	Age	City
4	Zara	24	Karachi

	Name	Age	City
1	Sara	30	Karachi

3 Hina 28 Islamabad

4 Zara 24 Karachi

Name Age City

0 Ali 25 Lahore

3 Hina 28 Islamabad

4 Zara 24 Karachi

✓ Key Points

- Boolean indexing me True/False mask banta hai
- `df[condition]`, `loc[]`, `query()` se filtering ho sakti hai
- `&` (AND), `|` (OR), `~` (NOT) operators multiple conditions ke liye
- `isin()`, `between()`, `str.contains()` helpful special filters hain
- Always use parentheses `()` around conditions

Q: Boolean indexing me kya hota hai?

A: True/False values ke basis pe elements filter karte hain.

Interview Qs + Answers:

Q1: Boolean mask kya hota hai?

Ek boolean array jo batata hai kaunse elements select hone hain.

Q2: Filtering kaise karte hain?

Condition likh kar array me, jaise `a[a>10]`.

Q3: Boolean indexing me brackets lagana zaroori hai?

☞ Haan, multiple conditions combine karte waqt `()` lagana mandatory hota hai.

Q4: `&` aur `and` me kya farq hai?

☞ `&` element-wise operation karta hai (Pandas me), jabke `and` sirf single boolean values ke liye hota hai.

Q5: `query()` kab use karte hain?

☞ Jab string-based condition likhni ho (clean syntax ke liye).

Q6: Agar condition kisi row ke liye False ho to kya hota hai?

👉 Woh row result me include nahi hoti.

Interview-Style Questions:

- Boolean indexing kaise kaam karti hai Pandas me?
- loc[] vs query() me difference explain karo.
- isin() aur between() ka use case batao.
- Multiple conditions apply karte waqt &, |, ~ ka role kya hai?
- Pandas me condition-based filtering kyu fast hoti hai (vectorization)?

8. Stacking & Splitting:

Function ke Kaam

np.concatenate(): Arrays jodna

np.vstack(): Vertically jodna

np.hstack(): Horizontally jodna

np.split(): Equal parts me todna

np.hsplit() / np.vsplit(): Specific axis pe todna

np.concatenate() — Arrays jodna

English: Concatenate multiple arrays along an existing axis.

Roman Urdu: Do ya zyada arrays ko **saath jodta hai** (same shape along axis required).

Examples:

np.concatenate() — Arrays jodna:

```
a = np.array([1,2,3])
```

```
b = np.array([4,5,6])
```

```
print(np.concatenate((a,b)))
```

Output:

```
[1 2 3 4 5 6]
```

np.vstack() — Vertically jodna (row-wise):

Example:

```
x = np.array([1,2,3])  
  
y = np.array([4,5,6])  
  
print(np.vstack((x,y)))
```

Output:

```
[[1 2 3]  
  
 [4 5 6]]
```

np.hstack() – Horizontally jodna (column-wise)

Example:

```
x = np.array([[1],[2],[3]])  
  
y = np.array([[4],[5],[6]])  
  
print(np.hstack((x,y)))
```

Output:

```
[[1 4]  
  
 [2 5]  
  
 [3 6]]
```

np.split() – Equal parts me todna:

Example:

```
arr = np.array([1,2,3,4,5,6])  
  
print(np.split(arr, 3))
```

Output:

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

np.hsplit() – Horizontally todna (columns ke hisse):

Example:

```
m = np.array([[1,2,3,4],  
  
              [5,6,7,8]])  
  
print(np.hsplit(m, 2))
```

Output:

```
[array([[1, 2],  
       [5, 6]]),  
array([[3, 4],  
       [7, 8]])]
```

np.vsplit() – Vertically todna (rows ke hisse):**Example:**

```
m = np.array([[1,2,3,4],  
              [5,6,7,8]])  
  
print(np.vsplit(m, 2))
```

Output:

```
[array([[1, 2, 3, 4]]),  
array([[5, 6, 7, 8]])]
```

Interview Qs + Answers**Q: vstack aur hstack me kya farq hai?**

A: vstack vertical (rows add karta hai), hstack horizontal (columns add karta hai).

Q: split ka use kya hai?

A: Large array ko chote arrays me todne ke liye.

🔺 9. Advanced Topics:**⚡ Vectorization:**

Definition: Jab for-loops ke bajaye pure arrays par ek sath operation lagaya jaye to ise vectorization kehte hain.

Benefit: Speed fast ho jaati hai kyunki loop Python me nahi chalta, backend C code me chalta hai.

⚙️ Common Methods / Tools for Vectorization

Method / Concept	Purpose
Element-wise operators (+ - * /)	Pure arrays par bina loop ke operation
Universal Functions (ufuncs)	Jaise np.add, np.subtract, np.sqrt, np.exp, etc.
Broadcasting	Different shape arrays par operation allow karta hai
Vectorized conditionals (np.where)	Element-wise conditional operation
Aggregation functions (np.sum, np.mean, etc.)	Fast element-wise reduce operations
Avoid Python loops	For-loops ki jagah direct array operations use karna

Examples with Outputs

Sample arrays

```
a = np.array([1,2,3,4,5])
```

```
b = np.array([10,20,30,40,50])
```

1. Element-wise operator

```
print(a + b)
```

Output:

```
[11 22 33 44 55]
```

2. Universal function (ufunc)

```
print(np.sqrt(a))
```

Output:

```
[1.      1.41421356 1.73205081 2.      2.23606798]
```

3. Broadcasting

```
m = np.array([[1],[2],[3]])  
  
n = np.array([10,20,30])  
  
print(m + n)
```

Output:

```
[[11 21 31]  
 [12 22 32]  
 [13 23 33]]
```

4. Vectorized conditional

```
print(np.where(a > 2, 'High', 'Low'))
```

Output:

```
['Low' 'Low' 'High' 'High' 'High']
```

5. Aggregation

```
print(np.sum(b))
```

Output:

```
150
```

✓ Key Points

- Vectorization se loops avoid hote hain → code fast hota hai
- NumPy ke ufuncs + broadcasting vectorization enable karte hain
- CPU-level optimized operations → performance improve hoti hai
- Cleaner, concise, and Pythonic code

Interview

Q1: Vectorization kya hoti hai aur NumPy me kaise kaam karti hai?

Answer:

Vectorization ka matlab hai array ke har element par bina loop lagaye operation apply karna. NumPy internally C language optimized code use karta hai jo operations ko parallel run karta hai — isliye bohot fast hota hai.

Q2: Vectorized operation aur loop-based operation me kya farq hai?

Answer:

Loop-based me har element pe alag se kaam hota hai (slow), jabke vectorized me poore array par ek sath operation hota hai (fast).

Example:

for loop vs $a + b$ (element-wise) → vectorized much faster.

Q3: np.where() ka role vectorization me kya hai?

Answer:

np.where() se hum if-else condition har element par bina loop ke apply kar sakte hain — isliye ye vectorized conditional tool hai.

Q4: Universal functions (ufuncs) kya hoti hain?

Answer:

Ufuncs wo ready-made vectorized functions hain jo array ke har element par fast operation karte hain.

Q5: Broadcasting ka vectorization se kya relation hai?

Answer:

Broadcasting chhote array ko bada treat karke vectorized operation ko allow karti hai — bina loop likhe alag shape ke arrays pe operation ho jata hai.

Q6: Vectorization code ko fast kaise banati hai (internally)?

Answer:

Vectorization Python loop ko skip karke direct CPU-level C code me kaam karti hai, jo parallel instructions aur continuous memory blocks use karta hai — isliye bohot fast hoti hai.

Q7: Vectorization fast kyun hai?

Ans: Kyunki Python loop avoid hota hai, operations optimized C loops me execute hote hain.

 **Copy vs View**

Copy **View**

 **Explanation:**

Jab hum kisi existing array se naya array banate hain to wo ya to view (sirf reference hota hai) banti hai ya copy (naya independent array).

- View: Same data ko reference karta hai. Agar original change karo to view me bhi change hota hai.
- Copy: Data ki full independent copy banti hai. Original change karne se copy par koi asar nahi hota.

Method	Return Type	Description
view()	View	Same data ka reference banata hai
Slicing (a[:])	View	Slice lene par usually view milti hai
copy()	Copy	Naya independent data copy banata hai
np.copy(arr)	Copy	Independent array return karta hai

Examples with Outputs:

```
a = np.array([1,2,3,4,5])
```

1. View using slicing

```
v = a[1:4]
v[0] = 99
print("Original:", a)
print("View:", v)
```

Output:

```
Original: [ 1 99  3  4  5]
View: [99  3  4]
```

◆ **Note:** v[0] = 99 karne se a bhi change ho gaya — kyunki v ek **view** hai.

2. Copy using .copy():

```
c = a.copy()

c[0] = 111

print("Original:", a)

print("Copy:", c)
```

Output:

```
Original: [ 1 99  3  4  5]

Copy: [111 99  3  4  5]
```

◆ **Note:** `c[0] = 111` karne se `a` change nahi hua — kyunki `c` ek **independent copy** hai.

✓ Key Points:

- **View:** Same data reference, fast, memory-efficient
- **Copy:** Independent data, safe (original par koi asar nahi hota)
- `view()` ya slicing → view milti hai
- `copy()` ya `np.copy()` → copy milti hai
- Change in view → reflect in original
- Change in copy → no effect on original

Interview Questions with Answers:

Q: copy vs view me farq?

Ans: Copy new memory allocate karta hai, view same memory share karta hai.

🧠 Memory Layout (C vs F Order)

C-order (row-major): Elements row by row store hote hain (default)

F-order (column-major): Elements column by column store hote hain

Q: C vs F order me farq?

Ans: C-order row-wise traversal karta hai, F-order column-wise traversal karta hai.

Q: View aur Copy me main difference kya hai?

Answer:

View **original ke data ko refer** karta hai, jabke copy **naya data banati hai**.

Q: NumPy slicing se view milti hai ya copy?

Answer:

Slicing karne par **zyada tar cases me view milti hai**, copy nahi.

Q: Agar view me data change karen to kya original bhi change hota hai?

Answer:

Haan, view change karne se **original bhi change hota hai** (kyunki dono same data share karte hain).

Q: Kab copy banana better hota hai?

Answer:

Jab aap **original ko safe rakhna chahte hain**, tab copy() use karte hain.

Q: copy() slow kyu hoti hai view ke mukable?

Answer:

copy() **data ka naya memory block banati hai**, isliye view() ke comparison me **slow** hoti hai.

Structured Arrays

Definition: Jab NumPy array me har column ka data type different ho sakta hai (jaise table ke columns).

Explanation:

Structured arrays NumPy ka ek feature hain jisme hum ek hi array ke andar alag-alag data types ke named columns/fields store kar sakte hain — jaise table jisme har column ka alag data type hota hai.

Methods / Techniques:

Method / Attribute	Kaam (Purpose)
<code>np.array(..., dtype=structured_dtype)</code>	Structured array create karna
<code>dtype=[('field', type), ...]</code>	Fields define karne ka tarika
<code>arr['field_name']</code>	Specific field access karna
<code>arr.dtype.names</code>	Sab field names dekhna
<code>arr.astype(new_dtype)</code>	Structured array convert karna

Examples with Real Output:

1. Structured dtype define

```
dt = np.dtype([('name', 'U10'), ('age', 'i4'), ('marks', 'f4')])
```

2. Structured array create

```
students = np.array([
    ('Ali', 20, 85.5),
    ('Sara', 22, 91.0),
    ('Ahmed', 19, 78.2)
], dtype=dt)
```

```
print(students)
```

Output:

```
[('Ali', 20, 85.5) ('Sara', 22, 91.) ('Ahmed', 19, 78.2)]
```

3. Specific field access

```
print(students['name'])
```

Output:

```
['Ali' 'Sara' 'Ahmed']
```

4. Access multiple fields

```
print(students[['name','marks']])
```

Output:

```
[('Ali', 85.5) ('Sara', 91.) ('Ahmed', 78.2)]
```

5. Get all field names

```
print(students.dtype.names)
```

Output:

```
('name', 'age', 'marks')
```

✓ Key Points:

- Structured arrays me **mixed data types** allowed hain
- Har element ek **record (row)** hoti hai jisme multiple named fields hote hain
- Fields ko **column ke naam se access** kiya jata hai
- Useful for **table-like or record-like data**
- Normal NumPy arrays se zyada **flexible**, lekin slightly **slower**

Interview Style Questions with Answers

Q1: Structured arrays kya hoti hain?

Answer:

Structured arrays ek hi array me **alag data types ke multiple named columns** store karne ka tariqa hain.

Q2: Normal array aur structured array me kya difference hai?

Answer:

Normal arrays me **sirf ek hi data type** hota hai, jabke structured arrays me **multiple data types allowed** hote hain.

Q3: Structured array me fields ko kaise access karte hain?

Answer:

Column/field ka naam likh ke access karte hain — arr['name'] jaise.

Q4: Structured array banate waqt fields ka data type kaise set karte hain?

Answer:

Structured array banate waqt dtype me **field ka naam aur uska type** define karte hain.

Q5: Structured arrays kis tarah ke data ke liye useful hoti hain?

Answer:

Jab **har row me alag-alag types ke multiple attributes/columns** store karne ho — jaise student records (name, age, marks).

Q6: Structured array ka use kya hai?

Ans: Tabular mixed-type data ko efficiently store karne ke liye.

Masked Arrays:

Definition: Jab kuch data missing ya invalid ho, to unhe mask kar dete hain (ignore karte hain operations me).

Explanation:

Masked arrays wo arrays hoti hain jisme kuch elements ko temporarily chhupa (mask) diya jata hai, taake calculations me unhe ignore kiya ja sake — jaise missing ya invalid data.

Ye specially NaN ya corrupt data handle karne ke liye useful hoti hain.

Common Methods / Attributes:

Method / Attribute	Kaam (Purpose)
<code>np.ma.array(data, mask=...)</code>	Masked array create karna
<code>masked_array.data</code>	Actual data access karna
<code>masked_array.mask</code>	Mask (True/False) access karna
<code>np.ma.masked_where(condition, data)</code>	Condition-based masking
<code>np.ma.masked_equal(data, value)</code>	Specific value ko mask karna
<code>masked_array.filled(value)</code>	Masked values ko replace karke filled array lena
<code>masked_array.compressed()</code>	Sirf unmasked elements return karna

Examples with Real Output:

```
import numpy as np
```

```
import numpy.ma as ma
```

1. Basic masked array

```
data = np.array([1, 2, 999, 4, 5])
```

```
m = ma.array(data, mask=[0, 0, 1, 0, 0]) # 999 ko mask kar diya
```

```
print(m)
```

Output:

```
[1 2 -- 4 5]
```

2. Mask using condition

```
arr = np.array([10, -5, 7, -3, 9])
```

```
masked = ma.masked_where(arr < 0, arr)
```

```
print(masked)
```

Output:

```
[10 -- 7 -- 9]
```

3. Mask using masked_equal

```
x = np.array([1, 0, 3, 0, 5])
```

```
masked2 = ma.masked_equal(x, 0)
```

```
print(masked2)
```

Output:

```
[1 -- 3 -- 5]
```

4. filled() → replace masked with given value

```
print(masked2.filled(-1))
```

Output:

```
[ 1 -1  3 -1  5]
```

5. compressed() → only unmasked data

```
print(masked2.compressed())
```

Output:

[1 3 5]

✓ Key Points:

- Masked arrays allow **ignoring unwanted/missing values**
- Masked elements are shown as --
- Calculations automatically **skip masked values**
- mask=True → value hidden
- Use filled() to replace masked values
- Use compressed() to get only valid data

💬 Interview Questions with Answers:

Q1: Masked arrays kya hoti hain?

Answer:

Masked arrays wo arrays hain jisme **selected elements ko ignore kar diya jata hai** taake calculations me count na hon.

Q2: Masked arrays ka use kab hota hai?

Answer:

Jab data me **NaN, missing ya galat values** hon jo calculations me include nahi karni.

Q3: Masked array me -- ka kya matlab hota hai?

Answer:

-- ka matlab hai **ye value temporarily chhupai gayi hai** (ignore ho rahi hai).

Q4: filled() method kya karta hai?

Answer:

filled() **masked values ko replace karke ek normal array** return karta hai.

Q5: compressed() method kya karta hai?

Answer:

compressed() sirf **valid (unmasked) values** return karta hai.

Q6: Masked array ka use?

Ans: Jab missing/invalid values ko ignore karte hue calculation karni ho. 📎 15. NaN & Inf Handling

Function ke Kaam

`np.isnan()` NaN check

`np.isinf()` Inf check

`np.nan_to_num()` NaN replace

`np.isfinite()` Finite values check

🔥 Memory Layout in NumPy (C-order vs F-order):

📖 Explanation (Roman Urdu + English)

NumPy me multi-dimensional arrays memory me store hone ka tarika do tareeqon ka hota hai:

- C-order (row-major): Row-wise data store hota hai (pehle row ke saare elements memory me ek saath)
- F-order (column-major): Column-wise data store hota hai (pehle column ke saare elements memory me ek saath)

Ye layout performance aur loop traversal speed par asar daal sakti hai.

⚙️ Methods / Parameters:

Method / Parameter	Kaam (Purpose)
<code>order='C' / order='F'</code>	Array banate waqt ya reshape karte waqt layout set karna
<code>arr.flags['C_CONTIGUOUS']</code>	Check if array is stored in C-order
<code>arr.flags['F_CONTIGUOUS']</code>	Check if array is stored in F-order
<code>np.asfortranarray(arr)</code>	Convert array to F-order
<code>np.copy(order='F')</code>	Copy array in F-order

💻 Examples with Output

Create 2D array in default (C-order)

```
a = np.array([[1,2,3],  
              [4,5,6]], order='C')
```

```
print(a)

print("C-order:", a.flags['C_CONTIGUOUS'])

print("F-order:", a.flags['F_CONTIGUOUS'])
```

Output:

```
[[1 2 3]

 [4 5 6]]

C-order: True

F-order: False

# Create same array in F-order

b = np.array([[1,2,3],

              [4,5,6]], order='F')
```

```
print(b)

print("C-order:", b.flags['C_CONTIGUOUS'])

print("F-order:", b.flags['F_CONTIGUOUS'])
```

Output:

```
[[1 2 3]

 [4 5 6]]
```

C-order: False:

F-order: True

Convert existing array to F-order

```
c = np.asfortranarray(a)

print(c.flags['F_CONTIGUOUS'])
```

Output:

True

✓ Key Points:

- **C-order:** row-wise memory layout (default in NumPy)
- **F-order:** column-wise memory layout (Fortran style)
- flags attribute se layout check kar sakte hain
- order='C' ya order='F' set karke layout control kar sakte hain
- Performance optimization me helpful hoti hai (cache efficiency)

💬 Interview Questions (Roman Urdu me)

Q1: C-order aur F-order me basic farq kya hai?

Jawab:

C-order me data **row by row** memory me store hota hai, jabke F-order me data **column by column** memory me store hota hai.

Q2: NumPy me default memory layout konsa hota hai?

By default **C-order (row-major)** hota hai.

Q3: Layout check karne ka tareeqa kya hai?

`arr.flags['C_CONTIGUOUS']` ya `arr.flags['F_CONTIGUOUS']` se check karte hain.

Q4: F-order me array banane ka tareeqa kya hai?

`np.array(..., order='F')` ya `np.asfortranarray(arr)` use karte hain.

Q5: Memory layout performance ko kaise affect karta hai?

Agar loop traversal ka order aur memory layout match kare to **cache access fast hoti hai**, isliye **speed improve hoti hai**.

The End